Hopscotch: Review Moderation

What learners build, and how each review moves through it

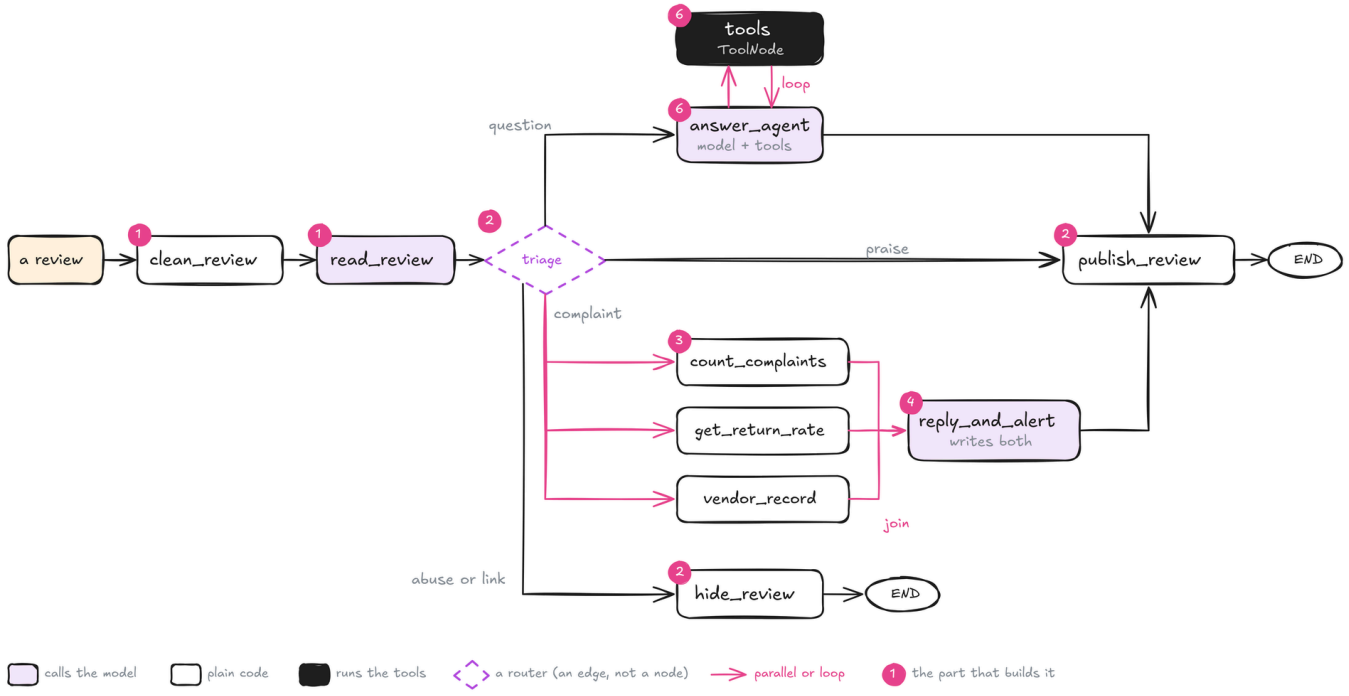
The problem

Hopscotch, a kids' clothing store, gets about **2,000 reviews a day**. People read them by hand, a week late. So:

- abuse and phone numbers stay on the site for days
- parents' questions go unanswered, and they buy elsewhere
- a bad batch is noticed only after hundreds of returns

Learners build one LangGraph graph that handles every review in minutes.

The graph



Purple boxes call the model. White boxes are plain Python. The dark box runs tools. The dashed diamond is a router. Pink arrows run at the same time, or go round in a loop. The pink numbers show which part builds each piece.

Every review takes one of four lanes

Each review is cleaned and read first. Then the router sends it down one lane:

Hide
Abuse or a spam link. The review is held back for a person to check.

Publish
Praise. The review goes live on the product page as it is.

Question
The model looks up the facts it needs with tools, then answers. The review goes live with the answer.

Complaint
Plain code fetches 3 facts at the same time. Then the model writes a public reply and an alert for the quality team.

The one rule behind every design choice: the model reads and writes; plain code makes the decisions. Which lane a review takes is a rule, so it is code. Writing a friendly reply is language, so it is the model.

Follow each test review

R-1 "Lovely soft fabric, my son wears it every day."
clean → read: praise → publish
Goes live

R-2 "Machine washable? Is 4-5Y in stock?"
clean → read: question → the agent asks for the care and stock tools together → answers
Goes live with an answer

R-7 "My daughter is 108 cm: which size, and is it in stock?"
clean → read: question → the agent asks for the size chart → then the stock for 4-5Y → answers
Goes live with an answer (2 trips round the loop)

R-3 "The colours bled after the very first wash."
clean → read: complaint → 3 lookups at once (6 complaints, 16% returns, vendor has 2 alerts) → reply and alert
Goes live with a reply; alert sent

R-4 "One strap button came off after a week."
clean → read: complaint → 3 lookups at once (1 complaint, 3% returns) → reply and alert
Goes live with a reply; alert sent

R-5 "Worst shoes ever, you idiots. Call me on 98200 11223."
clean (the phone number is hidden first) → read: abusive → hide
Held back

R-6 "Great deals on kids wear at http://cheap-..."
clean → read → the link rule catches it → hide
Held back

The nodes, lane by lane

The start: every review

`clean_review` plain code
Hides phone numbers and emails, so they never reach the model or go live.

`read_review` model
Reads the cleaned text and says what kind of review it is: praise, question, complaint or abusive.

`triage` router
Picks the lane, using simple rules in order. The first rule that matches wins.

The complaint lane

`count_complaints` plain code
How many similar complaints this product had in the last 14 days.

`get_return_rate` plain code
What share of this product's batch came back.

`vendor_record` plain code
How many quality alerts this vendor already had in 90 days.

`reply_and_alert` model
Writes two things: a short public reply, and an alert for the quality team with the 3 facts.

The question lane

`answer_agent` model
The model, with 3 tools it can ask for: size chart, care instructions, stock. When it needs nothing more, it answers.

`tools` runs tools
LangGraph's ToolNode. It runs whatever tools the agent asked for and hands the results back.

The endings

`hide_review` plain code
Holds the review back for a person to check.

`publish_review` plain code
Puts the review live, with Hopscotch's reply or answer if there is one.

Why tools in one lane and not the other?

Complaints: plain code
Every complaint needs the **same 3 facts**. There is nothing to choose, so code fetches all 3, at the same time.

Questions: tools
Every question needs **different facts**, sometimes one after another (size chart, then stock). So the model chooses.

The 6 LangGraph ideas, and where they appear

State
One shared dict that travels through the graph. Each node reads it and returns only what it changed.
Here: the review's text, kind, facts, reply and answer all live in one ReviewState.

```
return {"kind": result.kind}
```

Node and edge
A node is a Python function. An edge says what runs next.
Here: clean_review always runs before read_review, so the model only sees cleaned text.

```
graph.add_edge("clean_review", "read_review")
```

Router
A function that looks at the state and returns the name of the next node.
Here: triage sends each review to its lane.

```
graph.add_conditional_edges("read_review",
triage, LANES)
```

Parallel and join
If a router returns a list, those nodes run at the same time. A join waits for all of them.
Here: the 3 lookups take about 0.5 s together, instead of 1.5 s in a row.

```
graph.add_edge(LOOKUPS, "reply_and_alert")
```

Tools
Functions the model can ask to run. The model only asks; ToolNode runs them.
Here: each question needs different facts, so the model chooses.

```
model_with_tools = llm.bind_tools(TOOLS)
```

Loop
An edge back to an earlier node. A cap stops it running forever.
Here: tools goes back to answer_agent until it has its facts (at most 4 trips).

```
graph.add_edge("tools", "answer_agent")
```

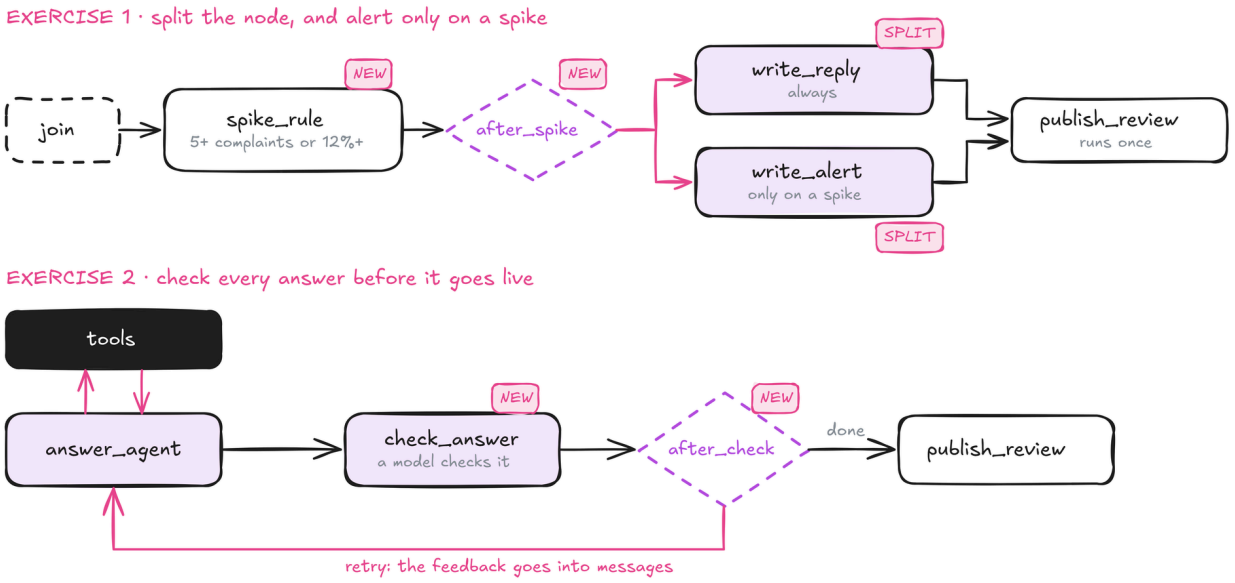
How the assignment builds it

About 80 minutes, in two sittings of 40. Each part tests its piece in a small graph first.

Part	Title	What learners do	Sitting
1	**Clean, then read**	the state and the first two nodes, one after the other	1st
2	**Triage**	the router and the four lanes	1st
3	**Three lookups at once**	parallel nodes and a join	1st
4	**The reply and the alert**	one model node that writes twice	1st
5	**Tools**	make tools and see the model ask for them	2nd
6	**The tool loop**	the agent, ToolNode and a loop with a cap	2nd
7	**Wire it all**	connect every node and run all 7 reviews	2nd

Extend it yourself (optional)

Two improvements learners build on their own, with no cells given. The finished graph runs before they start, and their new graph should run all 7 reviews after.



Exercise 1: alert only on a spike

Today every complaint raises an alert, so a real bad batch gets buried. Split the node into `write_reply` and `write_alert`, and add a plain-code `spike_rule` (5+ complaints or 12%+ returns). The alert is written only on a spike.

Done when only R-3 raises an alert.

Exercise 2: check every answer

Nothing checks an answer before it goes live. Add `check_answer`, a second model call with 3 rules. If an answer breaks one, the feedback goes back to the agent, which tries again (at most twice).

Done when a bad answer is rewritten.

Two more ideas to explore

A judge model

After the run, a second model scores every reply and answer. A decision model such as Jev (TypeSafe AI) gives a score with a confidence, cheaply.

A small model for hiding

A cheap, small model spots abuse and spam right after cleaning, so the bigger model is only paid for reviews that go live.